Module 1 — Introduction au Web

> **Objectif** : comprendre comment fonctionne le Web, identifier les acteurs (client, serveur, navigateur, hébergeur) et maîtriser le vocabulaire essentiel avant d'écrire la première ligne de code.

1. Internet ≠ Web

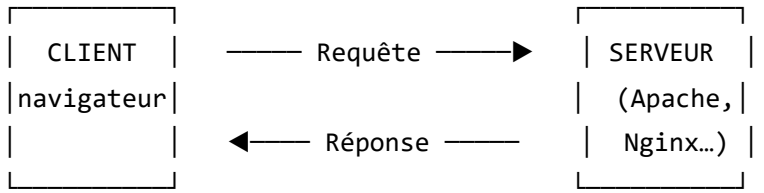
Beaucoup confondent les deux termes. La distinction est pourtant nette.

	Internet	Web
Nature	Réseau physique mondial d'ordinateurs interconnectés	Service qui utilise Internet pour partager des documents liés entre eux
Apparition	1969 (ARPANET)	1989 (Tim Berners-Lee, CERN)
Exemples d'usages	Web, e-mail, FTP, jeux en ligne, visioconférence, IoT	Sites web, applications web
Analogie	Le réseau routier	Le service de livraison qui utilise les routes

À retenir : le Web n'est qu'**une application** qui circule sur Internet. L'e-mail en est une autre.

2. Le modèle Client/Serveur

Tout le Web repose sur un dialogue entre deux acteurs.



- **Client** : votre navigateur (Chrome, Firefox, Safari, Edge…). Il **demande** une page.

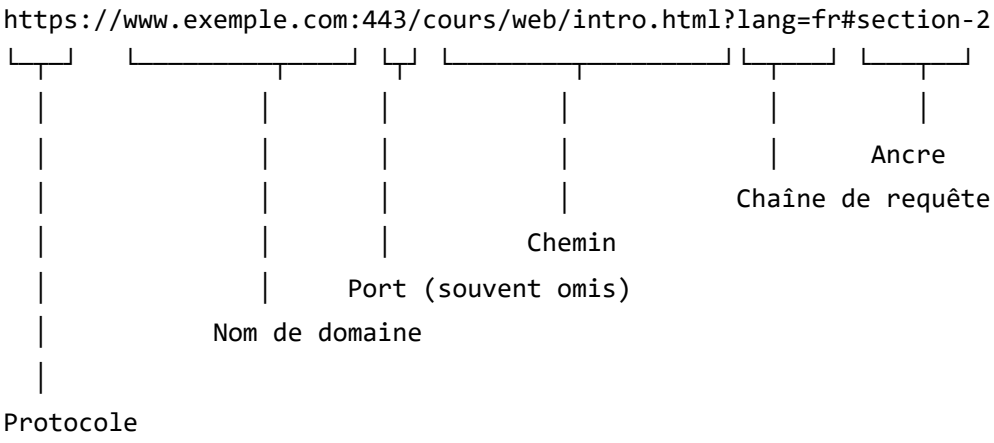
- **Serveur** : un ordinateur allumé en permanence qui **stocke** les fichiers du site et les **envoie** quand on les demande.

Analogie du restaurant

Restaurant	Web
Client à table	Navigateur
Serveur	Serveur HTTP (Apache, Nginx)
Carte du menu	URL d'une page
Commande passée	Requête HTTP
Plat servi	Réponse HTTP (HTML, CSS, images…)
Cuisine	Serveur applicatif + base de données

3. URL : l'adresse d'une ressource

Une **URL** (*Uniform Resource Locator*) identifie de façon unique une ressource sur le Web.



Élément	Rôle
Protocole	Règles de communication (`http` , `https` , `ftp` , `mailto` …)
Nom de domaine	Nom lisible du serveur (`google.com`)
Port	Numéro de "porte" sur le serveur (80 pour HTTP, 443 pour HTTPS — masqués par défaut)

Élément	Rôle
Chemin	Localisation du fichier sur le serveur
Query string	Paramètres envoyés au serveur (après `?`)
Ancre	Position dans la page (après `#`)

💡 **Astuce** : le `s` de `https` signifie *secure*. Les données sont chiffrées (TLS/SSL). Aujourd'hui, un site sans HTTPS est marqué "Non sécurisé" par les navigateurs.

4. DNS : l'annuaire d'Internet

Les ordinateurs communiquent par **adresses IP** (`172.217.20.142`), pas par noms. Le **DNS** (*Domain Name System*) traduit `google.com` en adresse IP.

```
Vous tapez "google.com"
        ↓
  Requête au DNS
        ↓
DNS répond : "172.217.20.142"
        ↓
Navigateur se connecte à cette IP
```

📚 **Pour aller plus loin** : tapez `nslookup google.com` dans un terminal pour voir le DNS en action.

5. HTTP : le langage du Web

HTTP (*HyperText Transfer Protocol*) est le protocole utilisé pour échanger des pages web.

Une requête HTTP

```
GET /cours/intro.html HTTP/1.1
Host: www.exemple.com
User-Agent: Mozilla/5.0
Accept: text/html
```

Une réponse HTTP

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 1024

<!DOCTYPE html>
<html>…</html>
```

Les méthodes HTTP principales

Méthode	Usage
`GET`	Récupérer une ressource (lire une page)
`POST`	Envoyer des données (soumettre un formulaire)
`PUT`	Mettre à jour une ressource entière
`PATCH`	Modifier partiellement une ressource
`DELETE`	Supprimer une ressource

Les codes de statut

Plage	Signification	Exemples
2xx	Succès	`200 OK` , `201 Created`
3xx	Redirection	`301 Moved Permanently` , `304 Not Modified`
4xx	Erreur côté client	`404 Not Found` , `403 Forbidden` , `401 Unauthorized`
5xx	Erreur côté serveur	`500 Internal Server Error` , `503 Service Unavailable`

⚠️ **À retenir** : `404` = la page n'existe pas. `500` = problème sur le serveur.

6. Front-end vs Back-end

Un site web complet est composé de **deux faces**.

	Front-end	Back-end
Aussi appelé	Côté client	Côté serveur
S'exécute	Dans le navigateur	Sur le serveur
Langages	HTML, CSS, JavaScript	PHP, Python, Node.js, Java, Ruby…
Outils	React, Vue, Angular	Laravel, Django, Express, Spring
Rôle	Affichage, interaction utilisateur	Logique métier, base de données, authentification
Visible ?	Oui (clic droit → "Afficher le code source")	Non (caché sur le serveur)

Full-stack = développeur qui maîtrise les deux côtés.

Le trio HTML / CSS / JavaScript

Langage	Rôle	Analogie maison
HTML	Structure, contenu	Le gros œuvre (murs, planchers)
CSS	Présentation, style	La décoration (peinture, meubles)
JavaScript	Comportement, interactivité	L'électricité (interrupteurs, électroménager)

7. Le navigateur en détail

Un navigateur est un **logiciel complexe** qui :

1. **Envoie** des requêtes HTTP.
2. **Reçoit** la réponse (HTML, CSS, JS, images…).
3. **Analyse** (parse) le HTML pour construire le **DOM** (*Document Object Model*).
4. **Applique** les styles CSS pour construire le **CSSOM**.
5. **Exécute** le JavaScript.
6. **Affiche** (render) le résultat à l'écran.

Les principaux navigateurs et leurs moteurs

Navigateur	Moteur de rendu	Moteur JavaScript
Chrome, Edge, Opera, Brave	**Blink**	V8
Firefox	**Gecko**	SpiderMonkey
Safari	**WebKit**	JavaScriptCore

💡 **Astuce** : testez vos sites sur **au moins deux moteurs différents** (par exemple Chrome + Firefox) pour éviter les surprises.

8. Les DevTools (outils de développement)

F12 (ou Ctrl+Maj+I) ouvre les *DevTools*, l'outil le plus important du développeur web.

Onglet	Utilité
Éléments / Inspector	Voir et modifier le HTML/CSS en direct
Console	Voir les erreurs JavaScript, exécuter du code
Réseau / Network	Voir les requêtes HTTP envoyées par la page
Sources	Déboguer le JavaScript (points d'arrêt)
Application	Inspecter cookies, localStorage, cache
Performance	Mesurer la rapidité d'affichage
Lighthouse	Audit qualité (performance, SEO, accessibilité)

🧪 **Mini-exercice** : ouvrez `https://www.wikipedia.org` , pressez F12, onglet **Réseau**, rechargez (Ctrl+R). Combien de requêtes HTTP la page a-t-elle déclenchées ? Combien de kilo-octets a-t-elle pesé ?

9. Cycle complet d'un affichage de page

Reprenons depuis le début, dans l'ordre :

1. **Vous tapez** `https://example.com` dans la barre d'adresse.

2. **DNS** traduit `example.com` en adresse IP.
3. **Connexion TCP** établie avec le serveur sur le port 443.
4. **Handshake TLS** (chiffrement HTTPS).
5. **Requête HTTP** `GET /` envoyée.
6. **Serveur répond** avec le HTML (statut `200 OK`).
7. **Navigateur parse** le HTML → construit le **DOM**.
8. En rencontrant `<link>` et `<img>`, il **redemande** les CSS, images, polices…
9. En rencontrant `<script>`, il **télécharge et exécute** le JavaScript.
10. **Rendu** final : pixels affichés à l'écran.

```
Tape URL → DNS → TCP → TLS → HTTP → HTML reçu →
Parse → DOM → Téléchargement CSS/JS/images →
CSSOM → Render Tree → Layout → Paint → Affichage
```

10. Hébergement et nom de domaine

Pour qu'un site soit accessible en ligne, il faut :

1. **Un nom de domaine** (10–15 €/an) — acheté chez un *registrar* (OVH, Namecheap, Gandi, Hostinger…).
2. **Un hébergement** — un serveur loué.

Types d'hébergement

Type	Pour qui ?	Coût indicatif
Mutualisé	Petits sites, débutants	2–10 €/mois
VPS *(Virtual Private Server)*	Sites moyens, applications web	5–50 €/mois
Dédié	Grosses applications, gros trafic	50–500+ €/mois
Cloud (AWS, Azure, GCP)	Scalabilité automatique	À l'usage
Statique gratuit	Portfolios, projets étudiants	0 € (GitHub Pages, Netlify, Vercel)

💡 Pour le projet portfolio du module, nous utiliserons **GitHub Pages** (gratuit).
💡 Pour le projet WordPress, nous utiliserons **Local by Flywheel** (local, gratuit) puis migrerons éventuellement vers un hébergement mutualisé.

11. Les standards du Web

Le Web est régi par des **organismes ouverts** qui publient les standards.

Organisme	Rôle
W3C *(World Wide Web Consortium)*	Standardise HTML, CSS, accessibilité
WHATWG	Maintient le standard "vivant" de HTML
IETF	Standardise les protocoles (HTTP, TLS…)
Ecma International	Standardise JavaScript (ECMAScript)

C'est grâce à ces standards qu'un site écrit en France fonctionne en Australie sur n'importe quel navigateur.

12. Accessibilité, SEO et performance : trois piliers transverses

Tout au long du module, gardez en tête trois exigences qui s'appliquent à **toutes** les pages web modernes.

Accessibilité (a11y)

Rendre les sites utilisables par **tous**, y compris les personnes handicapées (malvoyantes, daltoniennes, à mobilité réduite, sourdes…).

- Texte alternatif sur les images (`alt`).
- Contraste suffisant entre texte et fond.
- Navigation possible au clavier seul.
- Structure HTML sémantique.

SEO (*Search Engine Optimization*)

Optimiser un site pour qu'il soit bien classé par Google et Bing.

- Balises `<title>` et `<meta description>` pertinentes.
- Hiérarchie correcte des titres (`<h1>` → `<h6>`).
- URL lisibles.
- Contenu de qualité, mis à jour.
- Site rapide et mobile-friendly.

Performance

Un site lent perd ses visiteurs. Objectif : **moins de 3 secondes** pour s'afficher.

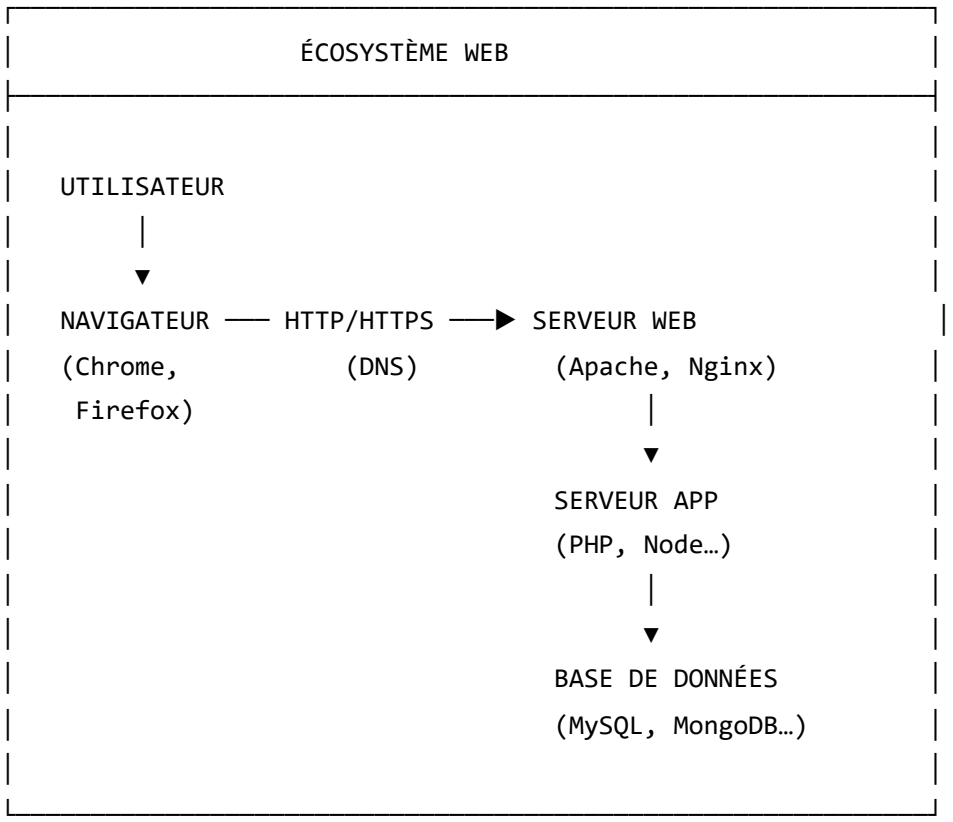
- Images optimisées (format WebP, dimensions adaptées).
- CSS et JavaScript minifiés.
- Hébergement géographiquement proche des visiteurs.
- Cache navigateur.

13. Vocabulaire essentiel à connaître

Terme	Définition courte
URL	Adresse d'une ressource web
HTTP / HTTPS	Protocole d'échange de pages web (S = sécurisé)
DNS	Annuaire qui traduit noms de domaine ↔ IP
DOM	Représentation arborescente d'une page HTML manipulable en JS
CSSOM	Équivalent du DOM pour les règles CSS
Front-end	Tout ce qui s'exécute dans le navigateur
Back-end	Tout ce qui s'exécute sur le serveur
API	Interface qui permet à des programmes de dialoguer
Responsive	Design qui s'adapte à toutes les tailles d'écran
Framework	Boîte à outils structurante (React, Vue, Laravel…)
CMS	*Content Management System* — gestionnaire de contenu (WordPress)

Terme	Définition courte
CDN	*Content Delivery Network* — réseau de serveurs qui distribuent le contenu plus vite

14. Récapitulatif visuel



15. Quiz d'auto-évaluation

1. Quelle est la différence entre Internet et le Web ?
2. Que signifie HTTPS ? Pourquoi est-ce important ?
3. Quel code HTTP indique qu'une page n'existe pas ?
4. Citez les trois langages du front-end et leur rôle.
5. À quoi sert le DNS ?
6. Qu'est-ce que le DOM ?
7. Quelle touche ouvre les DevTools ?
8. Donnez deux différences entre front-end et back-end.

> **Réponses** : 1. Internet = réseau, Web = service de pages liées · 2. HTTP sécurisé (chiffré TLS), protège les données · 3. 404 · 4. HTML (structure), CSS (style), JS (comportement) · 5. Traduire noms de domaine en IP · 6. Représentation arborescente de la page · 7. F12 · 8. Front = navigateur / Back = serveur ; Front = HTML/CSS/JS / Back = PHP/Node/Python…

Ressources

- 📚 MDN — Comment fonctionne le Web
- 📚 How HTTP Works (Mozilla)
- 📚 Can I Use — vérifier la compatibilité des fonctionnalités web
- 🎥 Vidéo Web Dev Simplified — How the web works (en anglais sous-titré)

➡️ **Module suivant** : 02 — HTML5